

Coding Interview Preparation Guide — Volume 3

Graphs (advanced), Greedy, and System/Object Design (Java 8+)

Target level: Mid-level software engineering interviews at companies such as Booking.com, Airbnb, Microsoft, Agoda, UiPath, and organizations of similar range.

About this volume

Volumes 1 and 2 covered the core patterns and went deep on binary search, tries, and DP. **Volume 3 closes the three biggest remaining gaps** for mid-level loops at these companies:

- **Graphs — Advanced.** The tools that trip candidates up because they didn't know the right one existed: **Union-Find** (disjoint sets), **Dijkstra** (shortest path with weights), topological sort on a real problem, and BFS-as-shortest-path.
- **Greedy.** Problems where making the locally-best choice provably gives the global optimum. The hard part is *recognizing* when greedy is safe — every problem here explains **why** the greedy choice works.
- **Design / Object-Oriented Design.** "Design a data structure that does X in $O(1)$." These test clean class design and picking the right combination of primitives (map + list, map + doubly-linked-list, etc.). **LRU Cache** is the classic and shows up constantly.

Every problem is **new** (nothing repeated from Volumes 1–2). Because several problems here are genuinely harder, this volume leans **more heavily on diagrams and multiple worked examples** — you'll often see two or three examples and a step-by-step trace.

What's inside

#	Section	Problems	Why it matters
1	Graphs — Advanced	6	Union-Find & Dijkstra are top "unknown tool" gaps
2	Greedy	5	Booking/Agoda love practical optimization
3	Design / OOD	4	LRU Cache & $O(1)$ structures are staples

The recognition checklist

1. **Union-Find?** Are you repeatedly asked "are these two connected?" or "how many groups?" while edges get added? Disjoint-set union with path compression is $O(\alpha(n))$ — effectively constant — per query.
2. **Dijkstra?** Shortest path in a graph with **non-negative weights**? Use a priority-queue-driven greedy expansion. (Unweighted → plain BFS is enough.)
3. **Topological sort?** A dependency ordering where "A must come before B"? Kahn's in-degree BFS, or DFS post-order.

4. **Greedy?** Can you argue that the locally-best pick never has to be undone later? If you can prove an **exchange argument**, greedy is safe. If not, it's probably DP.
 5. **Design?** List each operation and its required complexity first, then pick primitives that hit those bounds. $O(1)$ lookup $\rightarrow$ hash map; $O(1)$ ordered removal $\rightarrow$ doubly-linked list.
-

1. Graphs — Advanced

The core idea. A graph is usually stored as an **adjacency list**: a map (or array of lists) from each node to its neighbors, optionally with edge weights. Which algorithm you reach for depends on the *question being asked*: - **"Are these nodes connected / how many groups are there?"** → **Union-Find (DSU)**. It tracks connectivity without ever building or walking the graph again — great when edges arrive one at a time or you only care about grouping, not paths. - **"What's the shortest path, unweighted (or all edges weight 1)?"** → **BFS**. Level-by-level expansion guarantees the first time you reach a node is via the shortest path. - **"What's the shortest path with positive weights?"** → **Dijkstra** (priority queue, greedy: always expand the closest unfinished node). - **"Shortest path but weights can be negative, or there's a limit on the number of edges used?"** → **Bellman-Ford** (relax every edge, round by round — slower, but more flexible than Dijkstra because it doesn't assume a node is "done" after one visit). - **"Order these items given partial ordering constraints"** → build a graph of *dependencies* and run **topological sort** (Kahn's algorithm with indegrees).

Reach for it when: the problem mentions **connectivity/components** (Union-Find), a **grid or unweighted graph shortest path** (BFS), a **weighted shortest path** (Dijkstra / Bellman-Ford), or **ordering under constraints** (topological sort).

1.1 Number of Provinces

Problem. You are given n cities ($2 \leq n \leq 200$) and an $n \times n$ adjacency matrix `isConnected`, where `isConnected[i][j] == 1` if city i and city j are directly connected, and `isConnected[i][j] == 0` otherwise. The matrix is symmetric (`isConnected[i][j] == isConnected[j][i]`) and `isConnected[i][i] == 1` for every city. A **province** is a maximal group of cities connected directly or indirectly — i.e. a connected component of the graph the matrix describes. Return the total number of provinces.

- **Constraints:** $1 \leq \text{isConnected}[i][j] \leq 1$ (binary values only), matrix is always square and symmetric, no self-loops matter (diagonal is always 1).
- **Clarifying assumptions:** confirm the matrix is guaranteed symmetric (it is, per the problem) and that every city belongs to exactly one province, including cities with no connections at all (a province of size 1).
- **Why it's tricky:** it looks like a matrix problem, but it's really "count connected components," and the interviewer is testing whether you reach for Union-Find/DFS instead of trying to reason about the matrix directly.

Example.

```
Input:  isConnected = [[1,1,0],
                      [1,1,0],
                      [0,0,1]]
Output: 2           (cities 0-1 form one province, city 2 is its own province)
```

Intuition (diagram). This is exactly "count connected components." **Union-Find** (a.k.a. Disjoint Set Union / DSU) tracks this with two arrays: `parent[i]` = the node `i` currently points to, and `rank[i]` = a rough tree-height hint used to keep trees shallow. Initially every node is its own root (its own province). Each `1` in the matrix triggers a `union`, merging two provinces into one. At the end, the number of distinct roots = the number of provinces.

Forest **before** any unions (every node is its own root):

```
(0)      (1)      (2)
parent: [0, 1, 2]  <- each city points to itself
```

Forest **after** `union(0, 1)` (root 0 absorbs root 1, equal rank so pick 0 arbitrarily):

```
(0)              (2)
 |
(1)
parent: [0, 0, 2]  <- 1 now points to 0; 2 provinces remain: {0,1} and {2}
```

Path compression, visually. Suppose unions chained up a tall tree: `union(1,2)`, `union(2,3)`, `union(3,4)` left this before any `find` call:

```
1 <- 2 <- 3 <- 4      (arrows point from child to parent; root = 1)
```

Calling `find(4)` walks up to the root (`1`) and then **rewires every node it passed through to point directly at that root**, so the *next* `find` is $O(1)$:

```
  1
 / | \
2  3  4
```

All three nodes (2, 3, 4) now point directly at root `1` — the next `find(4)` is a single hop instead of a three-hop walk.

Approach (step by step)

Key idea / invariant: every node's `parent` pointer always leads, in one or more hops, to the root of its component; two nodes are in the same province exactly when `find(a) == find(b)`. The algorithm never explicitly builds or walks the adjacency graph — it only merges sets.

1. Initialize `parent[i] = i` for every city (each city starts as its own province of one).
2. For every pair `(i, j)` with `isConnected[i][j] == 1`, call `union(i, j)`:
3. `find(i)` walks up `parent` pointers until it reaches a node that is its own parent (the root), compressing the path as it goes so future lookups are faster.
4. If `find(i) != find(j)`, attach the smaller-rank root under the taller-rank root (or either one, if ranks are equal, bumping the winner's rank by one). This is why the trees stay shallow — you always attach the shorter tree to the taller one.
5. After processing every pair, scan all cities once and count how many satisfy `find(i) == i` (i.e. are their own root) — that count is the number of provinces.

Worked trace on the example (isConnected = [[1,1,0],[1,1,0],[0,0,1]]): - Start: parent = [0, 1, 2] . - Pair (0,1) : isConnected[0][1] == 1 → find(0)=0, find(1)=1, different roots → union → parent = [0, 0, 2] . - Pair (0,2) : isConnected[0][2] == 0 → skip. - Pair (1,2) : isConnected[1][2] == 0 → skip. - Final scan: find(0)=0 (root), find(1)=0 (not a root), find(2)=2 (root) → 2 roots → **2 provinces**, matching the expected output.

The reusable Union-Find (DSU) class

Path compression (flatten on find) + union by rank (attach the shorter tree under the taller one) keep every operation close to O(1) amortized. We'll reuse this class in 1.2.

```
class UnionFind {
    private final int[] parent;
    private final int[] rank;

    public UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i; // every node starts as its own root
        }
    }

    public int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]); // path compression: point straight at the root
        }
        return parent[x];
    }

    // returns false if x and y were already connected (i.e. this edge is redundant)
    public boolean union(int x, int y) {
        int rootX = find(x), rootY = find(y);
        if (rootX == rootY) {
            return false; // same root -> already in the same set
        }
        if (rank[rootX] < rank[rootY]) {
            parent[rootX] = rootY;
        } else if (rank[rootX] > rank[rootY]) {
            parent[rootY] = rootX;
        } else {
            parent[rootY] = rootX;
            rank[rootX]++; // trees were equal height, so the new one grows
        }
        return true;
    }
}
```

Brute force

DFS/BFS from every unvisited city, marking its whole province, and count how many times you start a fresh search.

```

public int findCircleNumBrute(int[][] isConnected) {
    int n = isConnected.length;
    boolean[] visited = new boolean[n];
    int provinces = 0;
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            provinces++;
            dfs(isConnected, visited, i);
        }
    }
    return provinces;
}

private void dfs(int[][] isConnected, boolean[] visited, int city) {
    visited[city] = true;
    for (int neighbor = 0; neighbor < isConnected.length; neighbor++) {
        if (isConnected[city][neighbor] == 1 && !visited[neighbor]) {
            dfs(isConnected, visited, neighbor);
        }
    }
}

```

Optimized (Union-Find)

Union every connected pair, then count distinct roots.

```

public int findCircleNum(int[][] isConnected) {
    int n = isConnected.length;
    UnionFind uf = new UnionFind(n);
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (isConnected[i][j] == 1) {
                uf.union(i, j);
            }
        }
    }
    int provinces = 0;
    for (int i = 0; i < n; i++) {
        if (uf.find(i) == i) { // i is a root -> it's the head of its own province
            provinces++;
        }
    }
    return provinces;
}

```

Version	Time	Space
Brute force (DFS)	$O(n^2)$	$O(n)$
Union-Find	$O(n^2 \cdot \alpha(n)) \approx O(n^2)$	$O(n)$

1.2 Redundant Connection

Problem. You are given a graph that started as a tree with n nodes (labeled $1..n$) and then had exactly one **extra edge** added, which created exactly one cycle. `edges` is the list of n edges, given in the order

they were added, each `edges[i] = [u, v]`. Find the edge that can be removed so the remaining graph is again a valid tree on all `n` nodes. If more than one edge could be removed to make it a tree, return the one that occurs **last** in the input.

- **Constraints:** `n == edges.length`, `1 <= n <= 1000`, `1 <= u, v <= n`, `u != v`, no duplicate edges, and the resulting graph is always connected (a tree plus one edge).
- **Clarifying assumptions:** confirm edges are undirected, and that the "extra" edge is guaranteed to exist (the input is never already a valid tree).
- **Why it's tricky:** the natural instinct is to find the cycle first and then figure out which edge to drop; Union-Find flips this around — process edges in order and the very first edge that *fails* to union two different sets is provably the answer, no explicit cycle-finding needed.

Example.

```
Input: edges = [[1,2],[1,3],[2,3]]
Output: [2,3]
```

Intuition (diagram). Add edges one at a time using the same `UnionFind` from 1.1. As long as an edge connects two **different** components, `union` succeeds — it's a legitimate tree edge. The moment an edge connects two nodes that are **already in the same set** (`find(u) == find(v)`), that edge is the one creating the cycle — return it immediately.

Building the forest edge by edge:

```
edge [1,2]: union(1,2) -> different roots, merge
    (1)
    |
    (2)

edge [1,3]: union(1,3) -> different roots, merge
    (1)
   /  \
  (2) (3)

edge [2,3]: find(2) == find(3) == 1 already! <- cycle detected
           this edge [2,3] is the answer
```

Approach (step by step)

Key idea / invariant: while scanning edges left to right, the Union-Find structure always reflects "is everything processed so far still a tree (or forest)?" The invariant is that every `union` call so far has connected two *previously separate* components. The first edge where that invariant would be violated — i.e. `find(u) == find(v)` before the union — is necessarily the redundant one, because everything before it formed a valid forest and this edge is the first one that doesn't grow it.

1. Initialize a `UnionFind` over nodes `1..n`.
2. For each edge `[u, v]` in input order:
3. Call `find(u)` and `find(v)`.

4. If they're equal, `u` and `v` are already connected through earlier edges — adding this edge creates the cycle. Return it immediately (this also guarantees the *last* valid answer, since we scan in input order and stop at the first conflict).
5. Otherwise, union them and continue.
6. If no conflict is ever found (shouldn't happen per the problem's guarantee), there is no redundant edge.

Worked trace on the example (`edges = [[1,2],[1,3],[2,3]]`): - `[1,2]` : `find(1)=1` , `find(2)=2` , different → union → tree is `1-2` . - `[1,3]` : `find(1)=1` , `find(3)=3` , different → union → tree is `1-2` , `1-3` . - `[2,3]` : `find(2)=1` , `find(3)=1` , **same root already** → this edge is redundant → return `[2,3]` , matching the expected output.

No brute force is worth showing separately here — the naive way (for each edge, remove it and DFS-check if the rest is a tree, $O(n^2)$) is really just Union-Find done the slow way. We show the $O(n^2)$ naive version and the near-linear Union-Find version.

```
public int[] findRedundantConnectionBrute(int[][] edges) {
    int n = edges.length;
    for (int skip = n - 1; skip >= 0; skip--) {
        if (formsValidTree(edges, n, skip)) {
            return edges[skip];
        }
    }
    return new int[0];
}

private boolean formsValidTree(int[][] edges, int n, int skipIndex) {
    UnionFind uf = new UnionFind(n + 1); // nodes are 1-indexed
    for (int i = 0; i < edges.length; i++) {
        if (i == skipIndex) continue;
        if (!uf.union(edges[i][0], edges[i][1])) {
            return false; // still a cycle without this edge
        }
    }
    return true;
}
```

Optimized (Union-Find, single pass)

```
public int[] findRedundantConnection(int[][] edges) {
    UnionFind uf = new UnionFind(edges.length + 1); // nodes are 1-indexed
    for (int[] edge : edges) {
        if (!uf.union(edge[0], edge[1])) {
            return edge; // this edge connects an already-connected pair
        }
    }
    return new int[0]; // no redundant edge (shouldn't happen per problem)
}
```

Version	Time	Space
Brute force (try removing each edge)	$O(n^2)$	$O(n)$
Union-Find (single pass)	$O(n \cdot \alpha(n)) \approx O(n)$	$O(n)$

1.3 Network Delay Time

Problem. There are n network nodes labeled $1..n$. $times[i] = [u, v, w]$ means there is a **directed** edge from node u to node v with a signal-travel time of w . You send a signal starting from node k . Return the amount of time it takes for **all** n nodes to receive the signal (i.e. the time the *last* node receives it), or -1 if it's impossible for some node to ever receive the signal.

- **Constraints:** $1 \leq n \leq 100$, $1 \leq times.length \leq 6000$, $times[i].length == 3$, $1 \leq u, v \leq n$, $u \neq v$, $0 \leq w \leq 100$, no duplicate (u, v) pairs, $1 \leq k \leq n$.
- **Clarifying assumptions:** confirm all weights are non-negative (they are, per constraints) — this is exactly why Dijkstra is valid here; also confirm the graph may be disconnected, in which case the answer is -1 .
- **Why it's tricky:** it's a straightforward "shortest path from one source" problem, but candidates often forget that a node can be pushed onto the priority queue multiple times with different (stale) distances, and forget to guard against processing a stale entry.

Example.

```
Input: times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2
Output: 2
```

Intuition (diagram). This is "shortest path from one source to all other nodes" with positive weights — textbook **Dijkstra**. Keep a `dist[]` array (initialized to infinity except the source = 0) and a min-priority-queue of $(node, distance)$. Always expand the node with the smallest known distance; once popped, its distance is final (never revisited).

Weighted graph:

```
(1)
 ^
 | 1
(2) --1--> (3) --1--> (4)
```

Approach (step by step)

Key idea / invariant (greedy relaxation): Dijkstra's correctness relies on processing nodes in increasing order of finalized distance. The invariant is: *once a node is popped from the priority queue with distance d , d is guaranteed to be its true shortest distance from the source* — because every edge weight is non-negative, so no path found later (through a node with a larger distance) could ever be shorter. This is why Dijkstra fails if edges can be negative (see 1.4).

The stale-entry guard: because the same node can be pushed onto the priority queue more than once (once for every edge that improves its distance before it's finalized), the queue can contain multiple, increasingly outdated entries for one node. When you pop $(node, d)$, check $d > dist[node]$ first — if true, a better distance for `node` was already finalized by an earlier pop, so this entry is stale and must be skipped, otherwise you'd re-relax edges from a node using a worse distance than it actually has.

1. Build an adjacency list: `node -> list of (neighbor, weight)`.

2. Initialize `dist[node] = infinity` for all nodes except `dist[k] = 0`. Push `(k, 0)` onto a min-heap keyed by distance.
3. While the heap is non-empty, pop the entry with the smallest distance `(node, d)`.
4. If `d > dist[node]`, this is a stale entry — skip it (guard above).
5. Otherwise, `node`'s distance is now finalized at `d`. Relax every outgoing edge `(node → next, weight)`: if `d + weight < dist[next]`, update `dist[next]` and push `(next, dist[next])`.
6. After the heap empties, every reachable node has its true shortest distance in `dist[]`. If any node's distance is still infinity, it's unreachable → return `-1`. Otherwise return `max(dist[])` — the time for the *last* node to receive the signal.

Worked trace of `dist[]` and the priority queue (`k = 2`):

Step	Popped (node,dist)	dist[1]	dist[2]	dist[3]	dist[4]	PQ after
1	(2, 0)	1	0	1	INF	[(1,1),(3,1)]
2	(1, 1)	1	0	1	INF	[(3,1)]
3	(3, 1)	1	0	1	2	[(4,2)]
4	(4, 2)	1	0	1	2	[]

Answer = `max(dist[]) = max(1, 0, 1, 2) = 2`.

Brute force (Bellman-Ford: relax every edge, $n-1$ times)

No priority queue needed, but much slower — useful when weights could be negative (Dijkstra can't be trusted then, see 1.4).

```
public int networkDelayTimeBrute(int[][] times, int n, int k) {
    int[] dist = new int[n + 1];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[k] = 0;
    for (int round = 0; round < n - 1; round++) { // a shortest path has at most n-1 edges
        for (int[] t : times) {
            int u = t[0], v = t[1], w = t[2];
            if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }
    int maxDist = 0;
    for (int node = 1; node <= n; node++) {
        if (dist[node] == Integer.MAX_VALUE) return -1;
        maxDist = Math.max(maxDist, dist[node]);
    }
    return maxDist;
}
```

Optimized (Dijkstra with a min-heap)

```
public int networkDelayTime(int[][] times, int n, int k) {
    Map<Integer, List<int[]>> graph = new HashMap<>();
    for (int[] t : times) {
        graph.computeIfAbsent(t[0], key -> new ArrayList<>()).add(new int[] { t[1], t[2] })
    }

    int[] dist = new int[n + 1];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[k] = 0;

    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> Integer.compare(a[1], b[1]));
    pq.offer(new int[] { k, 0 });

    while (!pq.isEmpty()) {
        int[] cur = pq.poll();
        int node = cur[0], d = cur[1];
        if (d > dist[node]) continue;
        for (int[] edge : graph.getOrDefault(node, Collections.emptyList())) {
            int next = edge[0], weight = edge[1];
            if (d + weight < dist[next]) {
                dist[next] = d + weight;
                pq.offer(new int[] { next, dist[next] });
            }
        }
    }

    int maxDist = 0;
    for (int node = 1; node <= n; node++) {
        if (dist[node] == Integer.MAX_VALUE) return -1;
        maxDist = Math.max(maxDist, dist[node]);
    }
    return maxDist;
}
```

Version	Time	Space
Brute force (Bellman-Ford)	$O(V \cdot E)$	$O(V)$
Dijkstra (min-heap)	$O(E \log V)$	$O(V + E)$

1.4 Cheapest Flights Within K Stops

Problem. There are n cities labeled $0..n-1$ and a list of flights $\text{flights}[i] = [\text{from}, \text{to}, \text{price}]$ (directed, one-way). Given a starting city src , a destination city dst , and an integer k , return the cheapest price to travel from src to dst using **at most k stops** (i.e. at most $k + 1$ edges/flights). Return -1 if there is no such route within that limit.

- **Constraints:** $1 \leq n \leq 100$, $0 \leq \text{flights.length} \leq (n * (n - 1)) / 2$, $0 \leq \text{from}, \text{to} < n$, $\text{from} \neq \text{to}$, $1 \leq \text{price} \leq 10^4$, no duplicate flights, $0 \leq \text{src}, \text{dst}$, $k < n$, $\text{src} \neq \text{dst}$.
- **Clarifying assumptions:** confirm "stops" means intermediate cities, not edges — $k = 0$ means only a direct flight is allowed; confirm prices are always positive (they are, per constraints), which

is what makes Bellman-Ford's fixed round count safe to reason about.

- **Why it's tricky:** the natural first instinct is Dijkstra (it's a shortest-path problem), but the k -stops constraint breaks Dijkstra's core assumption — see below.

Example.

```
Input:  n = 3, flights = [[0,1,100],[1,2,100],[0,2,500]], src = 0, dst = 2, k = 1
Output: 200          (0 -> 1 -> 2, one stop, cost 100+100=200; cheaper than the direct 500)
```

Intuition (diagram).

```
(0) --100--> (1) --100--> (2)
  \          ^
   \-----500-----/
```

Why plain Dijkstra doesn't work here: Dijkstra permanently finalizes a node's distance the first time it's popped, on the assumption that "cheapest so far = cheapest ever." That assumption is only true for *unconstrained* shortest paths. Here a node might be reached cheaply but using too many stops, and reached again more expensively but within the stop budget — Dijkstra would have already thrown away the chance to revisit it. **Bellman-Ford** fixes this naturally: doing exactly $k + 1$ rounds of "relax every edge" models "at most $k + 1$ edges used," and a node's distance is allowed to improve across rounds.

Approach (step by step)

Key idea / invariant (K-stops relaxation): after completing round r (1-indexed) of "relax every edge once," `dist[node]` holds the cheapest price to reach `node` using **at most r edges** — no more, no less. Doing exactly $k + 1$ rounds therefore gives, for every node, the cheapest price using at most $k + 1$ edges, which is exactly "at most k stops." Unlike Dijkstra, a node's distance is deliberately allowed to be revisited and improved in a later round, because "cheaper but more edges" is a valid trade the problem allows.

1. Initialize `dist[node] = infinity` for all nodes except `dist[src] = 0`.
2. Repeat exactly $k + 1$ times (one round per allowed edge):
 - a. Take a **snapshot** of the current `dist[]` (call it `next`, a clone). This is essential: relaxations within the same round must all read the *previous* round's distances, otherwise a single round could chain two edges together and effectively simulate an extra edge for free, corrupting the "at most r edges" invariant.
 - b. For every flight $(u, v, price)$, if `dist[u]` is finite and `dist[u] + price < next[v]`, update `next[v] = dist[u] + price`.
 - c. Set `dist = next` and move to the next round.
3. After $k + 1$ rounds, `dist[dst]` is the cheapest price reachable within $k + 1$ edges. If it's still infinity, return `-1`.

Worked trace on the example (`src=0, dst=2, k=1`, so 2 rounds): - Start: `dist = [0, INF, INF]` . - Round 1 (models "at most 1 edge"): relax $(0,1,100) \rightarrow next[1] = 0+100 = 100$; relax $(1,2,100) \rightarrow dist[1]$ was still `INF` in the *pre-round* snapshot, so this doesn't fire yet; relax $(0,2,500) \rightarrow next[2] = 0+500 = 500. After round 1: `dist = [0, 100, 500]` . - Round 2 (models "at most 2 edges"): relax $(0,1,100) \rightarrow$ no improvement; relax $(1,2,100) \rightarrow dist[1]=100$, so `next[2] = min(500, 100+100) = 200`; relax $(0,2,500) \rightarrow$ no improvement. After round 2: `dist = [0, 100, 200]` . - `dist[dst] = dist[2] = 200`, matching the expected output.$

Brute force (DFS over all paths, capped at k+1 edges)

Exponential, but conceptually simple: try every path, stop extending once you've used more than `k + 1` edges.

```
public int findCheapestPriceBrute(int n, int[][] flights, int src, int dst, int k) {
    Map<Integer, List<int[]>> graph = new HashMap<>();
    for (int[] f : flights) {
        graph.computeIfAbsent(f[0], key -> new ArrayList<>()).add(new int[] { f[1], f[2] })
    }
    int[] best = { Integer.MAX_VALUE };
    dfs(graph, src, dst, k + 1, 0, best);
    return best[0] == Integer.MAX_VALUE ? -1 : best[0];
}

private void dfs(Map<Integer, List<int[]>> graph, int node, int dst,
                int edgesLeft, int costSoFar, int[] best) {
    if (node == dst) {
        best[0] = Math.min(best[0], costSoFar);
        return;
    }
    if (edgesLeft == 0) return;
    for (int[] edge : graph.getOrDefault(node, Collections.emptyList())) {
        if (costSoFar + edge[1] < best[0]) { // simple pruning
            dfs(graph, edge[0], dst, edgesLeft - 1, costSoFar + edge[1], best);
        }
    }
}
```

Optimized (Bellman-Ford, limited to k+1 rounds)

```
public int findCheapestPrice(int n, int[][] flights, int src, int dst, int k) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    for (int round = 0; round <= k; round++) { // at most k+1 edges total
        int[] next = dist.clone(); // snapshot: don't chain within a round
        for (int[] f : flights) {
            int u = f[0], v = f[1], price = f[2];
            if (dist[u] != Integer.MAX_VALUE && dist[u] + price < next[v]) {
                next[v] = dist[u] + price;
            }
        }
        dist = next;
    }
    return dist[dst] == Integer.MAX_VALUE ? -1 : dist[dst];
}
```

Version	Time	Space
Brute force (DFS all paths)	$O(E^k)$ worst case	$O(k)$ recursion depth
Bellman-Ford (k+1 rounds)	$O(k \cdot E)$	$O(n)$

1.5 Word Ladder

Problem. Given two words `beginWord` and `endWord`, and a dictionary `wordList`, find the length of the **shortest transformation sequence** from `beginWord` to `endWord` such that: (1) each step changes exactly one letter, and (2) every intermediate word produced must exist in `wordList`. Return `0` if no such sequence exists. The sequence length counts words, not transformations (e.g. `hit` → `hot` has length 2).

- **Constraints:** `1 <= beginWord.length <= 10`, `endWord.length == beginWord.length`, `1 <= wordList.length <= 5000`, all words consist of lowercase English letters only, all words in `wordList` are distinct, `beginWord != endWord`.
- **Clarifying assumptions:** confirm `beginWord` itself does **not** need to be in `wordList` (it's the starting point, given separately); confirm `endWord` **does** need to be in `wordList` to be reachable — if it isn't, the answer is immediately `0`.
- **Why it's tricky:** it doesn't look like a graph problem at first glance — the "graph" is implicit (an edge exists between two words iff they differ by one letter), and it has to be discovered/generated on the fly rather than given directly; candidates who don't spot this often try slower string-matching approaches instead of BFS.

Example 1.

```
Input:  beginWord = "hit", endWord = "cog"
        wordList  = ["hot","dot","dog","lot","log","cog"]
Output: 5          (hit -> hot -> dot -> dog -> cog, 5 words = 4 transformations)
```

Intuition (diagram). Treat every word as a **node**; connect two words with an edge if they differ by exactly one letter. This is now "shortest path in an unweighted graph" — the job for **BFS**, expanding level by level so the first time you reach `endWord` is via the shortest sequence.

```
hit
 |
hot
 |-- dot -- dog --+
 |-- lot -- log --+
                    |
                    cog
```

BFS levels: `{hit}` → `{hot}` → `{dot, lot}` → `{dog, log}` → `{cog}` — 5 levels, matching the expected output of 5.

Approach (step by step)

Key idea / invariant: BFS explores the implicit word graph level by level, where "level" = number of transformations taken so far. The queue at any moment holds exactly the words reachable in the current number of steps, and every word is marked visited (removed from the dictionary) the moment it's enqueued — never revisited — which is what guarantees the *first* time `endWord` is dequeued, it was reached via the shortest possible sequence.

1. Put every word in `wordList` into a set (the dictionary); if `endWord` isn't in it, return `0` immediately — it's unreachable.

2. Start a BFS queue containing just `beginWord` , with `steps = 1` (counting `beginWord` itself as the first word in the sequence).
3. Process the queue **one full level at a time** (capture `queue.size()` before the inner loop, so you don't mix words from different levels):
4. For each word popped this level, if it equals `endWord` , return the current `steps` .
5. Otherwise, generate every possible one-letter variant of the word (swap each position to every other letter `a..z`). For each variant that exists in the dictionary, remove it from the dictionary (marks it visited) and enqueue it.
6. After finishing a level, increment `steps` and continue. If the queue empties without ever reaching `endWord` , return `0` .

Worked trace on Example 1 (dictionary starts as `{hot,dot,dog,lot,log,cog}`): - Level 1 (`steps=1`): `queue = {hit}` . Not `endWord` . Variants of `hit` in dict: `hot` . Remove `hot` , enqueue it. - Level 2 (`steps=2`): `queue = {hot}` . Not `endWord` . Variants of `hot` in dict: `dot` , `lot` . Remove and enqueue both. - Level 3 (`steps=3`): `queue = {dot, lot}` . Neither is `endWord` . Variants in dict: `dot -> dog` , `lot -> log` . Remove and enqueue both. - Level 4 (`steps=4`): `queue = {dog, log}` . Neither is `endWord` . Variants in dict: `dog -> cog` , `log -> cog` (only enqueued once, since it's removed after the first hit). - Level 5 (`steps=5`): `queue = {cog}` . `cog == endWord` → **return 5**, matching the expected output.

Example 2 (unreachable).

```
Input:  beginWord = "hit", endWord = "cog"
        wordList  = ["hot","dot","dog","lot","log"]      (note: "cog" is missing!)
Output: 0
```

Same graph as before, minus the `cog` node entirely:

```
hit
 |
hot
 |-- dot -- dog
 |-- lot -- log
```

`endWord` isn't even in the dictionary, so it's unreachable no matter how BFS explores — check for this up front and return `0` immediately.

Brute force (build the full graph, then search)

Compare every pair of words to decide if they're one letter apart — expensive, but shows the graph explicitly before searching it.

```

public int ladderLengthBrute(String beginWord, String endWord, List<String> wordList) {
    Set<String> words = new HashSet<>(wordList);
    words.add(beginWord);
    if (!words.contains(endWord)) return 0;           // endWord not reachable at all

    List<String> all = new ArrayList<>(words);
    Map<String, List<String>> graph = new HashMap<>();
    for (int i = 0; i < all.size(); i++) {            // O(N^2 * L) pairwise comparison
        for (int j = i + 1; j < all.size(); j++) {
            if (oneLetterApart(all.get(i), all.get(j))) {
                graph.computeIfAbsent(all.get(i), key -> new ArrayList<>()).add(all.get(j))
                graph.computeIfAbsent(all.get(j), key -> new ArrayList<>()).add(all.get(i))
            }
        }
    }

    Queue<String> queue = new LinkedList<>();
    Set<String> visited = new HashSet<>();
    queue.offer(beginWord);
    visited.add(beginWord);
    int steps = 1;
    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            String word = queue.poll();
            if (word.equals(endWord)) return steps;
            for (String next : graph.getOrDefault(word, Collections.emptyList())) {
                if (visited.add(next)) {
                    queue.offer(next);
                }
            }
        }
        steps++;
    }
    return 0;
}

private boolean oneLetterApart(String a, String b) {
    int diff = 0;
    for (int i = 0; i < a.length(); i++) {
        if (a.charAt(i) != b.charAt(i)) diff++;
    }
    return diff == 1;
}

```

Optimized (BFS, generate neighbors on the fly)

Skip building the graph entirely: for each word, try swapping every position to every letter `a..z` and check if the result is in the dictionary. This trades an $O(N^2)$ graph build for an $O(L \cdot 26)$ neighbor-generation step per word.


```

public int ladderLength(String beginWord, String endWord, List<String> wordList) {
    Set<String> dict = new HashSet<>(wordList);
    if (!dict.contains(endWord)) return 0; // unreachable, bail out early

    Queue<String> queue = new LinkedList<>();
    queue.offer(beginWord);
    dict.remove(beginWord); // double as the "visited" set

    int steps = 1;
    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        for (int i = 0; i < levelSize; i++) {
            String word = queue.poll();
            if (word.equals(endWord)) return steps;
            char[] chars = word.toCharArray();
            for (int pos = 0; pos < chars.length; pos++) {
                char original = chars[pos];
                for (char c = 'a'; c <= 'z'; c++) {
                    if (c == original) continue;
                    chars[pos] = c;
                    String candidate = new String(chars);
                    if (dict.remove(candidate)) { // in dict AND not visited yet
                        queue.offer(candidate);
                    }
                }
                chars[pos] = original; // restore before trying next position
            }
        }
        steps++;
    }
    return 0;
}

```

Version	Time	Space
Brute force (build graph + BFS)	$O(N^2 \cdot L)$	$O(N^2)$
Optimized (mutate + BFS)	$O(N \cdot L \cdot 26)$	$O(N \cdot L)$

1.6 Alien Dictionary

Problem. You are given a list of words, `words`, that is claimed to be sorted **lexicographically** according to the rules of some unknown alien language's alphabet (a permutation of some subset of lowercase English letters). Derive a valid ordering of the letters that is consistent with the given sort order. If the input is inconsistent — it contradicts itself so that no valid letter ordering exists — return `""`. If multiple valid orderings exist, any one of them is acceptable.

- **Constraints:** $1 \leq \text{words.length} \leq 100$, $1 \leq \text{words}[i].\text{length} \leq 100$, all characters are lowercase English letters, total length across all words fits comfortably in memory (roughly ≤ 2000 characters).
- **Clarifying assumptions:** confirm only **adjacent** words in the list need to be compared (if `words` is sorted, any ordering violation must show up between some consecutive pair); confirm ties are possible (two adjacent words can be identical, which yields no new edge).

- **Why it's tricky:** two separate invalid cases have to be handled, not just one: an outright **cycle** in the derived ordering constraints, and the subtler **prefix case** (a longer word sorted before one of its own prefixes), which produces no comparable characters at all and is easy to miss.

Example 1 (valid order).

```
Input: words = ["wrt","wrf","er","ett","rftt"]
Output: "wertf"
```

Intuition (diagram). Compare each **adjacent pair** of words letter by letter. The first position where they differ tells you one letter comes before another — that's a directed edge. Once the graph is built, a **topological sort** (Kahn's algorithm: repeatedly remove nodes with indegree 0) gives a valid global order.

Comparing adjacent pairs for Example 1:

```
wrt vs wrf -> differ at 't' vs 'f' => edge t -> f
wrf vs er  -> differ at 'w' vs 'e' => edge w -> e
er  vs ett -> differ at 'r' vs 't' => edge r -> t
ett vs rftt -> differ at 'e' vs 'r' => edge e -> r
```

Derived graph (a simple chain here) and indegrees:

```
w --> e --> r --> t --> f
```

letter	w	e	r	t	f
indegree	0	1	1	1	1

Approach (step by step)

Key idea / invariant: the alien ordering is really a "dependency" graph — an edge $a \rightarrow b$ means "a must come before b." A valid global order exists exactly when this graph is a **DAG** (no cycles); Kahn's algorithm both finds that order and detects cycles as a side effect: any letter never reaching indegree 0 is stuck inside a cycle.

Build-graph-then-topo-sort, in detail: 1. Collect every distinct letter across all words as a node, each starting at indegree 0. 2. For each **adjacent pair** of words (w_1, w_2) in the given order: - Scan both words position by position, up to the shorter word's length. - **Cycle-relevant edge case:** the moment you find the first differing character $c_1 \neq c_2$, add the edge $c_1 \rightarrow c_2$ (only if it's not already present, to avoid inflating indegree from duplicate edges) and **stop comparing this pair** — later differences in the same pair say nothing about global order, since a dictionary only needs the *first* differing letter to sort two words. - **Prefix edge case:** if you scan the entire shared length and find **no** differing character, no edge is created — but if w_1 is *longer* than w_2 in that case (e.g. $w_1 = "abc"$, $w_2 = "ab"$), the input is invalid: a longer word can never legally sort before its own prefix. Return `""` immediately. 3. Run Kahn's algorithm: seed a queue with every letter at indegree 0. Repeatedly pop a letter, append it to the result, and decrement the indegree of each of its neighbors; any neighbor that drops to indegree 0 joins the queue. 4. **Cycle detection:** if the final result contains fewer letters than were collected in step 1, some letters never reached indegree 0 — they're stuck in a cycle. Return `""`. Otherwise, the result is a valid alien alphabet order.

Kahn's algorithm on Example 1: only `w` starts at indegree 0. Pop `w`, decrement `e`'s indegree to 0, pop `e`, decrement `r`'s indegree to 0, pop `r`, then `t`, then `f`. Order = `w,e,r,t,f` = `"wertf"` — matches the expected output.

Example 2 (invalid — cycle, plus the prefix edge case).

```
Input: words = ["z","x","z"]
Output: ""      (invalid: z and x each demand the other comes first)
```

Comparing adjacent pairs:

```
z vs x -> differ at 'z' vs 'x' => edge z -> x
x vs z -> differ at 'x' vs 'z' => edge x -> z
```

Derived graph is a 2-cycle:

```
z ----> x
^         |
|         |
+-----+
```

letter	z	x
indegree	1	1

No letter has indegree 0, so Kahn's queue starts **empty** — the topological sort can't even begin, and the final order has fewer letters than the alphabet. That mismatch is exactly how you detect a cycle: **return ""**.

The other invalid case — prefixes. If one word is a longer version of the previous one, e.g. `["abc", "ab"]`, comparing letter by letter over their shared length finds **no difference at all** (`a==a`, `b==b`), so no edge is created — but `"abc"` still can't legally come before `"ab"` (a valid dictionary always lists a prefix before its extensions). This must be checked explicitly: if no differing character is found and the earlier word is **longer** than the later one, the input is invalid.

Brute force (try every permutation of the alphabet)

Generate every ordering of the unique letters and check whether it's consistent with all adjacent word pairs. Correct, but factorial time.

```

public String alienOrderBrute(String[] words) {
    Set<Character> charSet = new TreeSet<>();
    for (String w : words) {
        for (char c : w.toCharArray()) charSet.add(c);
    }
    List<Character> chars = new ArrayList<>(charSet);
    String result = permute(chars, 0, words);          // try every permutation (factorial)
    return result == null ? "" : result;
}

private String permute(List<Character> chars, int k, String[] words) {
    if (k == chars.size()) {
        char[] order = new char[chars.size()];
        for (int i = 0; i < chars.size(); i++) order[i] = chars.get(i);
        return isValidOrder(words, order) ? new String(order) : null;
    }
    for (int i = k; i < chars.size(); i++) {
        Collections.swap(chars, k, i);
        String result = permute(chars, k + 1, words);
        if (result != null) return result;
        Collections.swap(chars, k, i);                // backtrack
    }
    return null;
}

private boolean isValidOrder(String[] words, char[] order) {
    Map<Character, Integer> rank = new HashMap<>();
    for (int i = 0; i < order.length; i++) rank.put(order[i], i);
    for (int i = 0; i < words.length - 1; i++) {
        if (compare(words[i], words[i + 1], rank) > 0) return false;
    }
    return true;
}

private int compare(String a, String b, Map<Character, Integer> rank) {
    int n = Math.min(a.length(), b.length());
    for (int i = 0; i < n; i++) {
        int cmp = Integer.compare(rank.get(a.charAt(i)), rank.get(b.charAt(i)));
        if (cmp != 0) return cmp;
    }
    return Integer.compare(a.length(), b.length());    // shorter (prefix) must come first
}

```

Optimized (build graph + Kahn's topological sort)

```
public String alienOrder(String[] words) {
    Map<Character, Set<Character>> graph = new HashMap<>();
    Map<Character, Integer> indegree = new HashMap<>();
    for (String w : words) {
        for (char c : w.toCharArray()) {
            graph.putIfAbsent(c, new HashSet<>());
            indegree.putIfAbsent(c, 0);
        }
    }

    for (int i = 0; i < words.length - 1; i++) {
        String w1 = words[i], w2 = words[i + 1];
        int minLen = Math.min(w1.length(), w2.length());
        boolean foundDiff = false;
        for (int j = 0; j < minLen; j++) {
            char c1 = w1.charAt(j), c2 = w2.charAt(j);
            if (c1 != c2) {
                if (graph.get(c1).add(c2)) { // avoid double-counting the same
                    indegree.put(c2, indegree.get(c2) + 1);
                }
                foundDiff = true;
                break;
            }
        }
        if (!foundDiff && w1.length() > w2.length()) {
            return ""; // prefix edge case: "abc" before
        }
    }

    Queue<Character> queue = new LinkedList<>();
    for (char c : indegree.keySet()) {
        if (indegree.get(c) == 0) queue.offer(c);
    }
    StringBuilder order = new StringBuilder();
    while (!queue.isEmpty()) {
        char c = queue.poll();
        order.append(c);
        for (char next : graph.get(c)) {
            indegree.put(next, indegree.get(next) - 1);
            if (indegree.get(next) == 0) queue.offer(next);
        }
    }

    return order.length() == indegree.size() ? order.toString() : ""; // shorter -> a cycle
}
```

Version	Time	Space
Brute force (all permutations)	$O(U! \cdot N \cdot L)$	$O(U)$
Kahn's topological sort	$O(N \cdot L + U + E)$	$O(U + E)$

2. Greedy

The core idea. At every step you make the choice that looks best *right now*, and you never go back to reconsider it. This only works when the problem has a property called the **exchange argument**: you can prove that any solution which deviates from the greedy choice can be transformed into one that's at least as good by swapping in the greedy choice — meaning a locally optimal pick never needs to be undone later. Contrast this with **DP**, where a local choice *might* need to be revisited because it interacts with future choices in a way that isn't monotonic — DP explores (or memoizes) every branch and lets an optimization step pick the winner, whereas greedy commits immediately and never looks back. If you can't articulate why undoing a choice is never beneficial, you probably need DP instead.

Reach for it when: the problem asks for a **min/max count**, a **feasibility check** ("can you reach the end?"), or an **interval / scheduling** decision, *and* you can convince yourself that the locally best choice is never wrong.

2.1 Jump Game

Problem. Given an integer array `nums` of length `n`, where `nums[i]` represents the **maximum** number of steps you may jump forward from index `i` (you may jump anywhere from `1` to `nums[i]` steps, or not move if `nums[i] == 0`), and starting at index `0`, return `true` if you can reach index `n - 1` by any sequence of jumps, or `false` otherwise.

- **Constraints:** `1 <= nums.length <= 10^4`; `0 <= nums[i] <= 10^5`. Values are non-negative (a `0` means you cannot move forward from that index unless you jumped past it already).
- **Clarifying assumptions:** you always start at index `0`; if `n == 1` you're already at the last index, so the answer is trivially `true`; jump lengths are inclusive maximums, not exact — you may jump any number of steps from `1` up to `nums[i]`.
- **Why it's tricky:** it looks like it needs exploring many jump choices (exponential), but only one number — the farthest index reachable so far — ever matters; recognizing that collapses the search to a single linear pass.

Example.

```
Input:  nums = [2, 3, 1, 1, 4]
Output: true      (0 -> 1 -> 4, or 0 -> 2 -> 3 -> 4)
```

Intuition (diagram). Walk left to right, tracking the **farthest index reached so far**. At each index `i`, if `i` is beyond that farthest reach, you're stuck — no earlier jump could have gotten you here, so nothing can save you. Otherwise, greedily extend the farthest reach by `i + nums[i]`. You never need to remember *which* jump got you the farthest, only *how far* — that's the exchange argument: any jump that reaches at least as far as another dominates it, so only the max matters.

```

idx:      0   1   2   3   4
val:      2   3   1   1   4

```

```

i=0: reach = max(0, 0+2) = 2
      [=====]
      i         r

```

```

i=1: reach = max(2, 1+3) = 4    (1 is within old reach, so it's safe to use)
      [=====]
      i         r

```

```

i=2..4: i never exceeds reach -> keep extending -> reach hits the end (idx 4)
result: true

```

Second example — getting stuck on a zero.

```

Input:  nums = [3, 2, 1, 0, 4]
Output: false

```

```

idx:      0   1   2   3   4
val:      3   2   1   0   4

```

```

i=0: reach = max(0, 0+3) = 3
      [=====]
      i         r

```

```

i=1: reach = max(3, 1+2) = 3    (no improvement)
i=2: reach = max(3, 2+1) = 3    (no improvement)
i=3: reach = max(3, 3+0) = 3    idx 3 is a dead zero, reach stuck at 3
      [=====]
      i

```

```

i=4: i(4) > reach(3) -> UNREACHABLE, stop early
result: false    (never even need to look at nums[4] = 4)

```

Why greedy works. The only thing that matters about a prefix of jumps is the single number "farthest index reachable" — two different jump sequences that reach the same farthest point are interchangeable for everything that comes after. So keeping the maximum reach and discarding the rest loses no information, which is exactly the exchange argument.

Approach (step by step)

Invariant: after processing index i , `farthest` holds the maximum index reachable using only jumps that start at or before i . This is exactly the exchange-argument quantity: any two ways of reaching the same or a smaller index are interchangeable, so we only ever need to remember the best one.

1. Initialize `farthest = 0` (you start at index `0` with zero jumps used).
2. Scan i from `0` to $n - 1$: a. If $i > \text{farthest}$, no jump sequence can have landed on i — return `false` immediately. b. Otherwise i is reachable, so update `farthest = max(farthest, i + nums[i])` — this is the greedy choice: always take the longest jump *available* from every index you can legally stand on, because a longer reach can never hurt and might be needed later.
3. If the loop finishes without ever finding $i > \text{farthest}$, every index (including $n - 1$) was reachable — return `true`.

Why the greedy choice is safe (exchange argument): suppose an optimal solution reaches index $n - 1$ using jumps that, at some prefix, reach only $k < \text{farthest}$. Since farthest is achievable using indices already visited, we could swap in whichever jump produced farthest without making anything downstream worse — the swapped solution can reach at least as far from that point on. So there is never a reason to prefer a smaller reach over farthest ; keeping only the maximum loses nothing.

Worked trace ($\text{nums} = [2, 3, 1, 1, 4]$): farthest starts at 0. At $i=0$: $0 \leq 0$, so $\text{farthest} = \max(0, 2) = 2$. At $i=1$: $1 \leq 2$, so $\text{farthest} = \max(2, 4) = 4$. At $i=2$: $2 \leq 4$, $\text{farthest} = \max(4, 3) = 4$. At $i=3$: $3 \leq 4$, $\text{farthest} = \max(4, 4) = 4$. At $i=4$: $4 \leq 4$, loop ends, return `true`.

Brute force

Explore every jump length from every index (exponential without memoization); shown here as top-down DP with memoization to make it tractable.

```
public boolean canJumpDP(int[] nums) {
    Boolean[] memo = new Boolean[nums.length];
    return canJumpFrom(nums, 0, memo);
}

private boolean canJumpFrom(int[] nums, int i, Boolean[] memo) {
    if (i >= nums.length - 1) return true;
    if (memo[i] != null) return memo[i];
    for (int step = nums[i]; step >= 1; step--) { // try every reachable index from i
        if (canJumpFrom(nums, i + step, memo)) {
            return memo[i] = true;
        }
    }
    return memo[i] = false;
}
```

Optimized (greedy)

Track the farthest reachable index in one left-to-right pass.

```
public boolean canJump(int[] nums) {
    int farthest = 0;
    for (int i = 0; i < nums.length; i++) {
        if (i > farthest) return false; // this index is unreachable
        farthest = Math.max(farthest, i + nums[i]); // greedily extend reach
    }
    return true;
}
```

Version	Time	Space
Brute force (memoized DP)	$O(n^2)$	$O(n)$
Greedy (farthest reach)	$O(n)$	$O(1)$

2.2 Jump Game II

Problem. Given an integer array `nums` of length `n` where `nums[i]` is the maximum number of steps you may jump forward from index `i`, and it is **guaranteed** that you can reach index `n - 1`, return the **minimum number of jumps** required to reach index `n - 1` starting from index `0`.

- **Constraints:** `1 <= nums.length <= 10^4`; `0 <= nums[i] <= 1000`; the last index is always reachable (unlike Jump Game I, feasibility is not in question here).
- **Clarifying assumptions:** if `n == 1` the answer is `0` jumps (you're already there); a jump can land on any index up to `nums[i]` steps ahead, not exactly `nums[i]`.
- **Why it's tricky:** the natural instinct is to greedily jump as far as possible from the *current* position, but that's not quite the right invariant — the correct greedy groups indices into "BFS levels" (indices reachable in exactly `k` jumps) and only commits to a jump when the current level is exhausted.

Example.

```
Input:  nums = [2, 3, 1, 1, 4]
Output: 2          (0 -> 1 -> 4)
```

Intuition (diagram). Think of it as BFS over "levels," where level `k` is the set of indices reachable in exactly `k` jumps. Track `currentEnd` (the boundary of the current level) and `farthest` (the best boundary achievable for the *next* level). When your scan index `i` reaches `currentEnd`, you've exhausted the current level, so you must jump — bump the jump count and set `currentEnd = farthest`.

```
idx:      0   1   2   3   4
val:      2   3   1   1   4

level 0 (jumps=0): currentEnd=0, farthest=0

i=0: farthest = max(0, 0+2) = 2
      i == currentEnd(0) -> jump! jumps=1, currentEnd=2
      [=====]
          i         e
      (e = new currentEnd; everything up to idx 2 is reachable in 1 jump)

i=1: farthest = max(2, 1+3) = 4
i=2: farthest = max(4, 2+1) = 4
      i == currentEnd(2) -> jump! jumps=2, currentEnd=4
      [=====]
              i         e
      (everything up to idx 4 is reachable in 2 jumps)

i=3: farthest = max(4, 3+1) = 4   (loop stops before i=4: no need to jump from the last inc
result: jumps = 2
```

Why greedy works. Within a level, only the single number `farthest` matters — whichever index in the current level can reach furthest defines the entire next level's boundary. Any other index in the level is dominated by it, so there's no reason to branch on each one individually (which is what naive BFS/DP would do); collapsing each level to its farthest reach is safe and is exactly the exchange argument again.

Approach (step by step)

Invariant: `[0, currentEnd]` is exactly the set of indices reachable within the current number of jumps (`jumps`), and `farthest` is the best `currentEnd` achievable for `jumps + 1` — i.e., the farthest any index inside the current level can reach.

1. Initialize `jumps = 0`, `currentEnd = 0` (level-0 boundary — just index `0`), and `farthest = 0`.
2. Scan `i` from `0` to `n - 2` (never need to jump *from* the last index):
a. Update `farthest = max(farthest, i + nums[i])` — record the best reach achievable by any index seen so far in the current level.
b. If `i == currentEnd`, the scan has consumed every index in the current level without finding the target, so a jump is mandatory: increment `jumps` and set `currentEnd = farthest` (commit to the best next-level boundary).
3. Return `jumps`.

Why the greedy choice is safe (exchange argument): among all indices in the current level, only the one that achieves `farthest` needs to be remembered — any index in the level that reaches less than `farthest` is strictly dominated (whatever the dominant index can reach, includes or beats what the weaker one can reach), so branching on it separately (as plain BFS would) can never produce a better answer. Delaying the jump decision until `i == currentEnd` is also safe: jumping earlier only shrinks the next level's boundary, which can only add jumps, never save one.

Worked trace (`nums = [2,3,1,1,4]`): `jumps=0, currentEnd=0, farthest=0`. `i=0`: `farthest=max(0,2)=2`; `i==currentEnd(0) -> jumps=1, currentEnd=2`. `i=1`: `farthest=max(2,4)=4`. `i=2`: `farthest=max(4,3)=4`; `i==currentEnd(2) -> jumps=2, currentEnd=4`. `i=3`: `farthest=max(4,4)=4` (no `i==currentEnd` trigger since `i=3 != 4`, and the loop bound `n-1=4` is excluded). Return `jumps = 2`.

Brute force

BFS over reachable indices, or bottom-up DP computing `minJumps[i]` for every index.

```
public int jumpDP(int[] nums) {
    int n = nums.length;
    int[] minJumps = new int[n];
    Arrays.fill(minJumps, Integer.MAX_VALUE);
    minJumps[0] = 0;
    for (int i = 0; i < n; i++) {
        if (minJumps[i] == Integer.MAX_VALUE) continue;
        for (int step = 1; step <= nums[i] && i + step < n; step++) {
            minJumps[i + step] = Math.min(minJumps[i + step], minJumps[i] + 1);
        }
    }
    return minJumps[n - 1];
}
```

Optimized (greedy BFS levels)

Advance `currentEnd / farthest` boundaries; jump when the scan hits the current boundary.

```

public int jump(int[] nums) {
    int jumps = 0, currentEnd = 0, farthest = 0;
    for (int i = 0; i < nums.length - 1; i++) { // no need to jump from the last index
        farthest = Math.max(farthest, i + nums[i]);
        if (i == currentEnd) { // exhausted this level, must jump
            jumps++;
            currentEnd = farthest;
        }
    }
    return jumps;
}

```

Version	Time	Space
Brute force (DP)	$O(n^2)$	$O(n)$
Greedy (BFS levels)	$O(n)$	$O(1)$

2.3 Gas Station

Problem. There are n gas stations arranged in a circle, numbered 0 to $n - 1$. You have a car with an unlimited fuel tank, starting empty. $gas[i]$ fuel units are available at station i , and it costs $cost[i]$ fuel to travel from station i to station $i + 1$ (wrapping around from $n - 1$ to 0). Return the index of the starting station from which you can travel around the entire circuit exactly once without the tank ever going negative, or -1 if no such starting station exists.

- **Constraints:** $n == gas.length == cost.length$; $1 \leq n \leq 10^5$; $0 \leq gas[i], cost[i] \leq 10^4$.
- **Clarifying assumptions:** it is guaranteed that if a solution exists, it is **unique** — you don't need to return all valid starts, just the one; the tank can never go negative at any point along the route, not just at the end.
- **Why it's tricky:** the naive approach simulates the full loop from every candidate start ($O(n^2)$), and it's not obvious why checking $sum(gas) \geq sum(cost)$ plus one linear pass is enough to guarantee correctness — that uniqueness argument is the crux of the problem.

Example.

```

Input: gas = [1, 2, 3, 4, 5]
       cost = [3, 4, 5, 1, 2]
Output: 3

```

Intuition (diagram). First check feasibility: a full loop is possible only if $sum(gas) \geq sum(cost)$. Then simulate one pass starting from index 0 , tracking the running tank balance $gas[i] - cost[i]$. Whenever the tank goes negative, stations $0..i$ are all disqualified as starts (any of them would also go negative by the same accumulated deficit), so reset the candidate start to $i + 1$ and the tank to 0 . The station right after the **global minimum** of the running total is the answer.

idx:	0	1	2	3	4
gas:	1	2	3	4	5
cost:	3	4	5	1	2
diff:	-2	-2	-2	+3	+3

start=0, tank=0

i=0: tank = 0 + (-2) = -2 tank<0 -> disqualify [0..0], start=1, tank=0

i=1: tank = 0 + (-2) = -2 tank<0 -> disqualify [1..1], start=2, tank=0

i=2: tank = 0 + (-2) = -2 tank<0 -> disqualify [2..2], start=3, tank=0

i=3: tank = 0 + (+3) = 3 ok, keep going

i=4: tank = 3 + (+3) = 6 ok, loop complete

running total of diff (for intuition; not required by the algorithm):

idx:	0	1	2	3	4
total:	-2	-4	-6	-3	0
			^		

global minimum at idx 2 -> answer = idx 3 (right after the minimum)

result: start = 3

Why greedy works. If starting at station `s` makes the tank go negative by the time you reach station `e`, then starting anywhere between `s` and `e` (inclusive) is strictly worse or equal — you'd arrive at each of those stations with a smaller or equal head start, so you'd hit the same wall. That lets you skip the whole disqualified range at once instead of re-testing each station individually, which is the exchange argument: no station in `[s, e]` can ever be the answer, so throwing them all away loses nothing. Combined with the `sum(gas) >= sum(cost)` feasibility check, exactly one valid start remains (the one right after the point of maximum deficit).

Approach (step by step)

Invariant: at every point in the scan, `start` is the smallest index such that the running tank total, restarted at `start`, has never gone negative from `start` up to the current index `i`. Any index strictly before `start` has already been proven impossible as a starting point.

1. Initialize `totalGas = totalCost = 0` (to check feasibility at the end), `tank = 0` (running balance since the current candidate `start`), and `start = 0`.
2. Scan `i` from `0` to `n - 1`: a. Accumulate `totalGas += gas[i]` and `totalCost += cost[i]`. b. Update `tank += gas[i] - cost[i]`. c. If `tank < 0`, every station from `start` to `i` is disqualified — reset `start = i + 1` and `tank = 0`, and continue scanning from there.
3. After the scan, return `start` if `totalGas >= totalCost`, else `-1`.

Why the greedy choice is safe (exchange argument): if the tank goes negative between `start` and `i`, then for any candidate `s'` with `start <= s' <= i`, the prefix sum starting at `s'` is the same running total minus a non-negative prefix (the part from `start` to `s'-1`), so it goes negative at least as early — `s'` can never be a valid start either. That justifies jumping straight to `i + 1` without individually testing every station in between.

Why the answer is unique when `sum(gas) >= sum(cost)`: think of the running total (over one full pass, without resetting) as a curve that ends exactly at `sum(gas) - sum(cost) >= 0` (since the trip is circular, the total surplus is fixed regardless of start). The station right after the **global minimum** of that curve is the only place where, if you started there, the curve — measured relative to that new starting height — never dips below zero again before wrapping back around to itself. Starting anywhere else means you'd

pass through that same minimum point while still carrying whatever deficit came before it, which is exactly the negative dip the algorithm detects and disqualifies. Since the minimum is a single point (ties can be broken consistently by taking the earliest), exactly one starting station survives — hence "unique."

Worked trace (gas=[1,2,3,4,5] , cost=[3,4,5,1,2]): running total of diff is -2, -4, -6, -3, 0. The minimum is -6 at idx 2, so the answer is idx 3 — exactly what the one-pass algorithm found by resetting start three times (to 1, then 2, then 3) before finally holding a non-negative tank through the rest of the loop.

Brute force

Try every station as a start and simulate the full loop.

```
public int canCompleteCircuitBrute(int[] gas, int[] cost) {
    int n = gas.length;
    for (int start = 0; start < n; start++) {
        int tank = 0;
        boolean ok = true;
        for (int step = 0; step < n; step++) {
            int i = (start + step) % n;
            tank += gas[i] - cost[i];
            if (tank < 0) { ok = false; break; }
        }
        if (ok) return start;
    }
    return -1;
}
```

Optimized (greedy)

One pass: track running tank and reset the candidate start on deficit.

```
public int canCompleteCircuit(int[] gas, int[] cost) {
    int totalGas = 0, totalCost = 0;
    int tank = 0, start = 0;
    for (int i = 0; i < gas.length; i++) {
        totalGas += gas[i];
        totalCost += cost[i];
        tank += gas[i] - cost[i];
        if (tank < 0) { // stations [start..i] are all disqualified
            start = i + 1;
            tank = 0;
        }
    }
    return totalGas >= totalCost ? start : -1;
}
```

Version	Time	Space
Brute force	$O(n^2)$	$O(1)$
Greedy (running tank)	$O(n)$	$O(1)$

2.4 Assign Cookies

Problem. You are given an array `g` where `g[i]` is the greed factor of child `i` (the *minimum* cookie size needed to satisfy that child) and an array `s` where `s[j]` is the size of cookie `j`. Each child can receive **at most one** cookie, and a child is content only if the cookie they receive has size `s[j] >= g[i]`. Return the maximum number of children you can make content.

- **Constraints:** `1 <= g.length, s.length <= 3 * 10^4`; `1 <= g[i], s[j] <= 2^31 - 1`.
- **Clarifying assumptions:** cookies not given out are simply unused (no penalty); a child with no suitable cookie is left unsatisfied (not an error case); arrays may be different lengths and may contain duplicates.
- **Why it's tricky:** it's a matching problem that could tempt exhaustive pairing, but sorting both sides and matching the least-greedy child to the smallest cookie that can satisfy them is provably optimal — the trap is not seeing why "waste the least capacity" is safe.

Example.

```
Input: g = [1, 2, 3], s = [1, 1]
Output: 1          (only one cookie is big enough for the least-greedy child, size 1 -> g=1)
```

Intuition (diagram). Sort both arrays. Walk the least-greedy child and the smallest cookie together: if the smallest remaining cookie satisfies the least-greedy remaining child, give it to them — that cookie was never going to satisfy anyone *less* greedy anyway, so using it on the least-greedy child wastes the least potential. If it's too small, that cookie can't satisfy *any* remaining child (they're all greedier), so discard it.

```
g:      1  2  3
s:      1  1

g_ptr=0, s_ptr=0
      g
      s
g[0]=1, s[0]=1 -> 1<=1 satisfied! content++, g_ptr=1, s_ptr=1

      g
      s
g[1]=2, s[1]=1 -> 1<2  cookie too small, discard cookie, s_ptr=2

s_ptr out of range -> stop
result: content = 1
```

Why greedy works. If the smallest cookie can satisfy the least-greedy child, giving it to anyone greedier would only waste it (a greedier child needs a bigger cookie, and this is the smallest one, so it can't help them either... but assigning it to the least-greedy child never blocks a better outcome, since any child it could satisfy, the least-greedy one also qualifies for). And if the smallest cookie can't satisfy even the least-greedy child, no larger child can use it, so it's dead weight — remove it. Both moves are safe swaps, the essence of the exchange argument.

Approach (step by step)

Invariant: at every step, `g[gi]` is the least-greedy *unassigned* child and `s[si]` is the smallest *unused* cookie; every child before `gi` has already been matched, and every cookie before `si` has already been

either used or correctly discarded as useless.

1. Sort `g` and `s` ascending.
2. Initialize two pointers `gi = 0` (next child to satisfy) and `si = 0` (next cookie to try), and `content = 0`.
3. While both pointers are in range: a. If `s[si] >= g[gi]`, this cookie satisfies the current least-greedy child: increment `content`, and advance `gi` (move to the next child). b. Advance `si` regardless — whether the cookie was used or too small, it's done being considered.
4. Return `content`.

Why the greedy choice is safe (exchange argument): consider any optimal assignment. If the smallest cookie `s[si]` is big enough for the least-greedy child `g[gi]`, then in the optimal assignment either `g[gi]` already gets some cookie `c`, or it gets nothing. If `c` is a different (necessarily larger-or-equal) cookie, swap: give `s[si]` to `g[gi]` and give `c` to whoever `s[si]` was originally assigned to (if anyone) — since `g[gi]` is the least-greedy child, `s[si]` also satisfies every other child that `s[si]`'s original owner could have needed, so the swap never breaks another assignment. If `s[si]` can't satisfy even `g[gi]` (the least-greedy child), it can't satisfy anyone, so discarding it costs nothing. Both moves preserve or improve the total content count, which is the exchange argument.

Worked trace (`g=[1,2,3]`, `s=[1,1]`, both already sorted): `gi=0, si=0 : s[0]=1 >= g[0]=1 -> content=1, gi=1, si=1. gi=1, si=1 : s[1]=1 < g[1]=2 -> cookie discarded, si=2. si=2 out of range -> loop ends. Return content = 1.`

Brute force

For each child in greed order, scan all remaining cookies for the smallest one that fits.

```
public int findContentChildrenBrute(int[] g, int[] cost) {
    int[] s = cost;
    Arrays.sort(g);
    boolean[] used = new boolean[s.length];
    int content = 0;
    for (int gi : g) {
        int bestIdx = -1;
        for (int j = 0; j < s.length; j++) {           // scan every cookie
            if (!used[j] && s[j] >= gi) {
                if (bestIdx == -1 || s[j] < s[bestIdx]) bestIdx = j;
            }
        }
        if (bestIdx != -1) { used[bestIdx] = true; content++; }
    }
    return content;
}
```

Optimized (sort + two pointers greedy)

Sort both arrays; advance in lockstep.

```

public int findContentChildren(int[] g, int[] s) {
    Arrays.sort(g);
    Arrays.sort(s);
    int gi = 0, si = 0, content = 0;
    while (gi < g.length && si < s.length) {
        if (s[si] >= g[gi]) { // cookie satisfies this child
            content++;
            gi++;
        }
        si++; // move on to the next cookie either way
    }
    return content;
}

```

Version	Time	Space
Brute force	$O(n \cdot m)$	$O(m)$
Greedy (sort + two pointers)	$O(n \log n + m \log m)$	$O(1)$ extra

2.5 Partition Labels

Problem. Given a string `s`, partition it into as many contiguous, non-overlapping substrings ("parts") as possible such that every occurrence of a given letter falls entirely within a single part (i.e., no letter is split across two parts). Return a list of the lengths of these parts, in order.

- **Constraints:** `1 <= s.length <= 500`; `s` consists only of lowercase English letters.
- **Clarifying assumptions:** the partition covering the whole string always exists (in the worst case, one part containing everything), so the answer is never empty; parts must be contiguous substrings, not arbitrary subsets of characters.
- **Why it's tricky:** it looks like it needs interval-merging over every letter's first/last occurrence, but a single left-to-right scan that only tracks one running boundary (the last occurrence of every character seen so far) is enough — recognizing that the boundary only needs to be checked against the *current* scan position is the key insight.

Example.

```

Input: s = "ababcbacadefegdehijhklij"
Output: [9, 7, 8]

```

Intuition (diagram). First record the **last occurrence index** of every character. Scan left to right, extending the current partition's end to `max(end, lastOccurrence[c])` for each character `c` you see. When the scan pointer `i` catches up to `end`, the partition is closed — every character seen so far is guaranteed to not reappear later.


```

s:      a b a b c b a c a d e f e g d e h i j h k l i j
idx:    0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23

last-occurrence map (partial): a->8 b->5 c->7 d->14 e->15 f->11 g->13
                                h->19 i->22 j->23 k->20 l->21

i=0 'a': end=max(0,8)=8
i=1 'b': end=max(8,5)=8
...(scan continues, end stays 8 until i catches up)...
i=8 'a': end=8, i==end -> CLOSE partition [0..8], length 9

|.....| |.....| |.....|
          len 9          len 7          len 8

result: [9, 7, 8]

```

Why greedy works. Once the scan index reaches the current partition's `end`, every character encountered so far has its *last* occurrence at or before `end` — by definition of the last-occurrence map — so none of them can leak into a later partition. Closing as early as possible (as soon as `i == end`) maximizes the number of partitions, and delaying the cut could only merge two valid partitions into one, never help — that's the exchange argument here: an earlier cut is never worse.

Approach (step by step)

Invariant: while scanning inside the current partition, `end` is the farthest index that any character seen so far in this partition is known to reappear at; the partition cannot be safely closed until the scan pointer `i` reaches `end`, because closing earlier would cut a character's last occurrence out of its own partition.

1. Precompute `lastOccurrence[c]` for every letter `c` in `s` by scanning once and always overwriting — after the scan, each entry holds that letter's last index.
2. Initialize `start = 0` (the start of the current partition) and `end = 0` (its current known boundary).
3. Scan `i` from `0` to `len(s) - 1`: a. Update `end = max(end, lastOccurrence[s[i]])` — the greedy step: the current partition must extend at least to the last place this character reappears. b. If `i == end`, no character seen since `start` can reappear after `i` (their last occurrences are all `<= end == i`), so it's safe to close: record length `end - start + 1`, then set `start = i + 1` for the next partition.
4. Return the recorded lengths.

Why the greedy choice is safe (exchange argument): suppose you tried to close a partition later than necessary, at some `end' > end` where `i == end` already held. The extra characters between `end` and `end'` never needed to be in this partition (nothing forced them in), so splitting them off — cutting at `end` instead — is always at least as good, and can only increase (never decrease) the total partition count. Symmetrically, you cannot close *earlier* than `end`, because some character seen since `start` is guaranteed to reoccur at index `end`, which would violate the "no letter split across two parts" rule. So `end` is both the earliest safe cut and the latest necessary one — cutting exactly there is optimal.

Worked trace (`s = "ababcbacadegefegdehijklj"`): scanning from `i=0`, `end` climbs to `8` because 'a' last occurs at index `8` (higher than 'b' 's last occurrence `5`). `end` stays at `8` through `i=1..7` since no character in that range has a later last-occurrence. At `i=8`, `i == end`, so the first partition closes with length `8 - 0 + 1 = 9`. The scan resets `start = 9` and repeats: `end` climbs to `15` (driven

by 'e' 's last occurrence), closes at `i=15` with length `7` . Finally `end` climbs to `23` (driven by 'j'), closes at `i=23` with length `8` . Result: `[9, 7, 8]` .

Brute force

For each candidate cut, verify no character crosses it by checking first/last occurrence overlaps directly (essentially interval merging over each letter's `[first, last]` span).

```
public List<Integer> partitionLabelsBrute(String s) {
    int n = s.length();
    int[] first = new int[26], last = new int[26];
    Arrays.fill(first, -1);
    for (int i = 0; i < n; i++) {
        int c = s.charAt(i) - 'a';
        if (first[c] == -1) first[c] = i;
        last[c] = i;
    }
    List<int[]> intervals = new ArrayList<>();
    for (int c = 0; c < 26; c++) {
        if (first[c] != -1) intervals.add(new int[] { first[c], last[c] });
    }
    intervals.sort((a, b) -> a[0] - b[0]);    // merge overlapping [first,last] intervals
    List<Integer> result = new ArrayList<>();
    int curStart = intervals.get(0)[0], curEnd = intervals.get(0)[1];
    for (int[] iv : intervals) {
        if (iv[0] > curEnd) {
            result.add(curEnd - curStart + 1);
            curStart = iv[0];
            curEnd = iv[1];
        } else {
            curEnd = Math.max(curEnd, iv[1]);
        }
    }
    result.add(curEnd - curStart + 1);
    return result;
}
```

Optimized (greedy, last-occurrence map)

Single pass, extend `end` , cut when `i == end` .

```
public List<Integer> partitionLabels(String s) {
    int[] lastOccurrence = new int[26];
    for (int i = 0; i < s.length(); i++) {
        lastOccurrence[s.charAt(i) - 'a'] = i;    // overwrite -> ends up last in
    }
    List<Integer> result = new ArrayList<>();
    int start = 0, end = 0;
    for (int i = 0; i < s.length(); i++) {
        end = Math.max(end, lastOccurrence[s.charAt(i) - 'a']);
        if (i == end) {    // no pending char crosses here
            result.add(end - start + 1);
            start = i + 1;
        }
    }
    return result;
}
```

Version	Time	Space
Brute force (interval merge)	$O(n \log n)$	$O(n)$
Greedy (last-occurrence scan)	$O(n)$	$O(1)$ extra (26-letter map)

3. Design / Object-Oriented Design

The core idea. Design problems don't ask you to find a clever trick — they ask you to build a small data structure that supports a fixed set of operations under a fixed time budget, usually $O(1)$ or $O(\log n)$ per call. The way to approach every one of these: first write down each operation and the time bound it must hit, *before* touching any code. Then pick primitives that hit those bounds — a **hash map** gives $O(1)$ lookup by key but has no order; pair it with something that *does* have order or structure: a **doubly linked list** for $O(1)$ insert/remove-at-either-end, an **array/ArrayList** for $O(1)$ random access by index, or a **bucket array** for $O(1)$ average-case hashing. The map tells you *where* something lives; the second structure tells you *how it's arranged*. Almost every "design X" interview question is this same combination wearing a different costume.

Reach for it when: the prompt says "**design a data structure that supports get/put/insert/delete/random — all in $O(1)$ [or $O(\log n)$]**" — that phrasing is the signal to stop and list operations + bounds before writing a single line.

3.1 LRU Cache

Problem. Design and implement a **Least Recently Used (LRU) cache** with a fixed positive `capacity`, given as a constructor argument. Support two operations, both required to run in $O(1)$: - `get(key)` — if `key` exists, return its value and mark it as the most recently used entry; otherwise return `-1`. - `put(key, value)` — if `key` exists, update its value and mark it as most recently used; if it doesn't exist, insert it as most recently used. If inserting pushes the number of entries over `capacity`, evict the **least recently used** entry first, before inserting.

Constraints (typical LeetCode bounds): - $1 \leq \text{capacity} \leq 3000$ - $0 \leq \text{key}, \text{value} \leq 10^4$ - At most $2 * 10^5$ calls total to `get` and `put`.

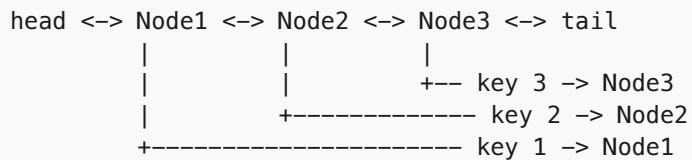
Clarifying assumptions — confirm with the interviewer: - "Used" means either a `get` or a `put` on that key — both refresh recency, not just reads. - Keys and values are non-negative ints (or generic types); no special "not found" sentinel other than `-1` is needed since the problem guarantees values are in that range.

Why it's tricky: no single built-in structure gives you both $O(1)$ lookup-by-key *and* $O(1)$ "what's the least recently used entry right now" — you need two structures that share state, kept perfectly in sync on every operation.

Example.

```
LRUCache cache = new LRUCache(2);    // capacity = 2
cache.put(1, "A");                    // cache: {1=A}           MRU->LRU: 1
cache.put(2, "B");                    // cache: {1=A, 2=B}       MRU->LRU: 2,1
cache.get(1);                         // 1 becomes most recent  MRU->LRU: 1,2
cache.put(3, "C");                    // capacity full -> evict LRU (2)
                                     // cache: {1=A, 3=C}           MRU->LRU: 3,1
cache.get(2);                         // 2 was evicted
```

Intuition (diagram). No single structure gives you both "find by key instantly" and "know what's oldest instantly." So use two structures pointing at *the same nodes*: - A **HashMap** for O(1) lookup — jump straight to a node without scanning. - A **doubly linked list** ordered by recency, **head** (most-recently-used) to **tail** (least-recently-used). Every touch moves a node to the head; eviction removes the tail. Both point at the same **Node** objects, so moving a node in the list doesn't touch the map at all — the map entry still points at the same object, wherever it now sits in the list.



The map's job is purely "given a key, hand me the **Node** pointer in O(1)." The list's job is purely "keep every node ordered by recency, and let me splice any node out or in near **head** in O(1)." Neither structure alone can do both.

Step-by-step for the example above (dummy **head** / **tail** sentinels omitted from the key list for brevity, shown only as anchors):

```

put(1,"A")  list: head <--> [1:A] <--> tail

put(2,"B")  list: head <--> [2:B] <--> [1:A] <--> tail
              ^ new node inserted right after head

get(1)      unlink [1:A], reinsert right after head:
            list: head <--> [1:A] <--> [2:B] <--> tail

put(3,"C")  size would become 3 > capacity(2), so evict first:
            list before evict: head <--> [1:A] <--> [2:B] <--> tail
                                      ^ node just before tail = LRU victim

            remove [2:B], map.remove(2)
            list after evict:  head <--> [1:A] <--> tail
            insert [3:C] right after head:
            list: head <--> [3:C] <--> [1:A] <--> tail

get(2)      map has no entry for 2 (it was evicted) -> return -1
  
```

Approach (step by step)

Key idea / invariant. Maintain the doubly linked list so it is always ordered by recency, most-recently-used immediately after **head**, least-recently-used immediately before **tail**. The map's only job is O(1) key -> **Node** lookup; the list's only job is O(1) reordering and O(1) eviction from the tail end. Every operation that "touches" a key ends by moving that key's node to be the very first real node (right after **head**).

1. **get(key)** : look up **key** in the map. If absent, return **-1** (O(1) map lookup, no list work). If present, unlink the node from its current position and reinsert it right after **head** (two O(1) pointer splices), then return **node.value**.
2. **put(key, value) — key already exists**: update **node.value**, then run the same "move to front" splice as in **get**. No eviction needed since the count didn't grow.

3. **put(key, value) — new key**: first check if `map.size() == capacity`. If so, evict: read `tail.prev` (the LRU node), unlink it from the list, and remove its key from the map — this is the $O(1)$ eviction trace below. Then create the new node, put it in the map, and insert it right after `head`.
4. **Why each step is $O(1)$** : the map gives direct pointer access (no scan to find a node by key), and a doubly linked list lets you unlink any node given only a pointer to it — you never walk the list to find `prev` / `next`, because the node already carries them. Using `head` / `tail` sentinel nodes means insert-after-head and remove-before-tail never need a null check for "is this the first/last real node" — that's what keeps the splice logic simple and branch-free.

Eviction trace (capacity 2, cache already has `1:A` most-recent, `2:B` least-recent, then `put(3,"C")`):

```
step 1  map.size()==2==capacity -> must evict
step 2  lru = tail.prev           -> lru is Node(2,"B")
step 3  remove(lru)               -> list: head <--> [1:A] <--> tail
step 4  map.remove(lru.key)        -> map no longer has key 2
step 5  fresh = Node(3,"C")
step 6  map.put(3, fresh)
step 7  insertAfterHead(fresh)    -> list: head <--> [3:C] <--> [1:A] <--> tail
```

Implementation (HashMap + hand-rolled doubly linked list)

```
public class LRUCache {
    private class Node {
        int key, value;
        Node prev, next;
        Node(int key, int value) { this.key = key; this.value = value; }
    }

    private final int capacity;
    private final Map<Integer, Node> map = new HashMap<>();
    private final Node head = new Node(-1, -1); // sentinel: most-recent side
    private final Node tail = new Node(-1, -1); // sentinel: least-recent side

    public LRUCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    public int get(int key) {
        Node node = map.get(key);
        if (node == null) return -1;
        moveToFront(node); // touched -> now most recently used
        return node.value;
    }

    public void put(int key, int value) {
        Node existing = map.get(key);
        if (existing != null) {
            existing.value = value;
            moveToFront(existing);
            return;
        }
        if (map.size() == capacity) {
            Node lru = tail.prev; // node just before tail sentinel = least recent
            remove(lru);
            map.remove(lru.key);
        }
        Node fresh = new Node(key, value);
        map.put(key, fresh);
        insertAfterHead(fresh);
    }

    // -- linked-list helpers, all O(1) --

    private void remove(Node node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    private void insertAfterHead(Node node) {
        node.next = head.next;
        node.prev = head;
        head.next.prev = node;
        head.next = node;
    }

    private void moveToFront(Node node) {
        remove(node);
        insertAfterHead(node);
    }
}
```

```
}
}
```

Operation	Required	Achieved
<code>get(key)</code>	$O(1)$	$O(1)$ — map lookup + $O(1)$ list splice
<code>put(key, value)</code>	$O(1)$	$O(1)$ — map lookup/insert + $O(1)$ list splice
Space	—	$O(\text{capacity})$

A shortcut worth *mentioning* in an interview: Java's `LinkedHashMap` already maintains access order internally (`new LinkedHashMap<>(cap, 0.75f, true)`), and overriding `removeEldestEntry` gives you an LRU cache in ~10 lines. It's a fine thing to name as a one-liner alternative, but implement the map + DLL version by hand — that's what's actually being tested.

3.2 Insert Delete GetRandom O(1)

Problem. Implement a `RandomizedSet` class supporting, all in **average $O(1)$** time: - `insert(val)` — if `val` is not already present, add it and return `true`; otherwise return `false` without changing the set. - `remove(val)` — if `val` is present, remove it and return `true`; otherwise return `false`. - `getRandom()` — return a random element from the current elements, each with equal probability. Only called when the set is non-empty.

Constraints (typical LeetCode bounds): - $-2^{31} \leq \text{val} \leq 2^{31} - 1$ - At most $2 * 10^5$ calls total to `insert`, `remove`, and `getRandom`. - There will be at least one element when `getRandom()` is called.

Clarifying assumptions — confirm with the interviewer: - Values are unique inside the set (it's a set, not a multiset) — duplicates never coexist. - "Uniformly random" means every currently-present element must have exactly equal probability of being returned — a random walk on some internal structure isn't good enough if that structure isn't uniformly indexable.

Why it's tricky: `getRandom()` in $O(1)$ forces you toward an array (index `rand.nextInt(n)` directly), but arrays make `remove(val)` $O(n)$ unless you avoid ever shifting elements — that's what the swap-with-last trick is for.

Example.

```
RandomizedSet set = new RandomizedSet();
set.insert(10);      -> true    // list: [10]           map: {10:0}
set.insert(20);      -> true    // list: [10, 20]        map: {10:0, 20:1}
set.remove(10);      -> true    // swap 10 with last (20), pop -> list: [20]
set.insert(30);      -> true    // list: [20, 30]        map: {20:0, 30:1}
set.getRandom();     -> 20 or 30, each with probability 1/2
```

Intuition (diagram). `getRandom()` in $O(1)$ needs **index-based random access**, which means the values must live in an **array/ArrayList**. But `remove(val)` in $O(1)$ needs to find *where* a value lives without scanning — that's a **HashMap**. The catch: removing from the *middle* of an `ArrayList` is $O(n)$ (everything after it shifts). The fix is the classic **swap-with-last** trick — swap the target with the last element, then remove the last element, which is $O(1)$.

Before remove(10):

index: 0 1
list: [10 , 20]
 ^target

map: {10:0, 20:1}

After remove(10):

index: 0
list: [20]
 ^ moved here from index 1

map: {20:0}

swap(0,1) -> [20,10] -> popBack() removes 10 from the end

Three-value example from a fresh set:

insert(10) insert(20) insert(30)
remove(20)

list: [10, 20, 30] map: {10:0,20:1,30:2}
target index = 1 (map.get(20))
swap list[1] with list[last=2]: [10, 30, 20]
update map[30] = 1 (it moved)
remove list's last element (20), map.remove(20)
final: list: [10, 30] map: {10:0, 30:1}

Approach (step by step)

Key idea / invariant. Keep the elements packed contiguously in `values[0..size-1]` with no gaps, and keep `indexOf` as the exact inverse mapping — `indexOf.get(v) == i` if and only if `values.get(i) == v`. Every mutation must restore both halves of that invariant together, or the map and array drift out of sync.

1. **insert(val)**: check `indexOf` for `val` in $O(1)$. If present, return `false` (no change). If absent, append `val` to the end of `values` ($O(1)$ amortized `ArrayList` append) and record `indexOf.put(val, values.size()-1)` — the new element's index is just "last slot."
2. **remove(val)**: look up `idx = indexOf.get(val)` in $O(1)$; if `null`, return `false`. Otherwise apply the **swap-with-last** trick so the removal happens at the *end* of the array, which is the only position an `ArrayList` can delete from in $O(1)$: a. Read `lastVal = values.get(size-1)`. b. Overwrite `values.set(idx, lastVal)` — the removed slot is now filled by what used to be the last element. c. Update `indexOf.put(lastVal, idx)` so the map reflects `lastVal`'s new position. d. Shrink the array by one (`values.remove(size-1)`) and `indexOf.remove(val)`. This is correct even when `val` is the last element — the swap becomes a harmless self-assignment before the shrink.
3. **getRandom()**: draw `rand.nextInt(values.size())` and index directly into `values`. Since every live element occupies exactly one contiguous slot with no gaps, every index is equally likely, which is exactly "uniformly random."
4. **Why $O(1)$** : no step scans anything proportional to the set's size — map lookups are $O(1)$, array append/shrink at the tail is $O(1)$, and the swap touches exactly one other element.

Worked trace — `remove(20)` on `values=[10,20,30]`, `indexOf={10:0,20:1,30:2}`:

```
idx = indexOf.get(20)    -> idx = 1
lastVal = values.get(2)  -> lastVal = 30
values.set(1, 30)        -> values = [10, 30, 30] (slot 1 overwritten, slot 2 still stale)
indexOf.put(30, 1)       -> indexOf = {10:0, 20:1, 30:1}
values.remove(2)         -> values = [10, 30] (stale last slot dropped)
indexOf.remove(20)       -> indexOf = {10:0, 30:1}
```

Implementation (HashMap + ArrayList, swap-remove)

```
public class RandomizedSet {
    private final Map<Integer, Integer> indexOf = new HashMap<>(); // value -> its index
    private final List<Integer> values = new ArrayList<>();
    private final Random rand = new Random();

    public boolean insert(int val) {
        if (indexOf.containsKey(val)) return false;
        indexOf.put(val, values.size());
        values.add(val);
        return true;
    }

    public boolean remove(int val) {
        Integer idx = indexOf.get(val);
        if (idx == null) return false;

        int lastVal = values.get(values.size() - 1);
        // move the last element into the removed slot (no-op if val IS the last element)
        values.set(idx, lastVal);
        indexOf.put(lastVal, idx);

        values.remove(values.size() - 1); // O(1): removing from the end of an ArrayList
        indexOf.remove(val);
        return true;
    }

    public int getRandom() {
        return values.get(rand.nextInt(values.size()));
    }
}
```

Operation	Required	Achieved
insert(val)	O(1) avg	O(1) — map + array append
remove(val)	O(1) avg	O(1) — map lookup + swap-with-last + pop
getRandom()	O(1)	O(1) — array index by rand.nextInt(size)
Space	—	O(n)

3.3 Design HashMap

Problem. Implement a `MyHashMap` class from scratch — no built-in `Map` / `HashMap` / `TreeMap` type allowed. Support: - `put(key, value)` — insert a new `(key, value)` pair, or update the value if `key` already exists. - `get(key)` — return the value mapped to `key`, or `-1` if `key` is not present. - `remove(key)` — remove the mapping for `key` if it exists (no-op otherwise). All three should run in **O(1) average time**.

Constraints (typical LeetCode bounds): - $0 \leq \text{key}, \text{value} \leq 10^6$ - At most 10^4 calls total to `put`, `get`, and `remove`.

Clarifying assumptions — confirm with the interviewer: - Keys are non-negative here, but a production-grade `hash(key)` should still handle negative keys correctly (via `Math.floorMod`, not `%`, since `%` can return a negative result in Java). - The bucket count is fixed for this exercise; mention resizing/rehashing as the real-world follow-up rather than implementing it, unless asked.

Why it's tricky: it's easy to write a hash map that's $O(1)$ on average but secretly relies on a built-in collection to do the hard part; the actual test is whether you understand *how* separate chaining achieves $O(1)$ — the bucket array plus per-bucket linear scan.

Example.

```
MyHashMap map = new MyHashMap();
map.put(1, 100);
map.put(2, 200);
map.get(1);      -> 100
map.get(3);      -> -1
map.put(2, 250); // update existing key
map.get(2);      -> 250
map.remove(2);
map.get(2);      -> -1
```

Intuition (diagram). This is what a hash map is doing *under the hood*. Keep a fixed-size **array of buckets**. `hash(key)` picks a bucket via `key % numBuckets`; every key that lands in the same bucket is chained together (a small linked list, or here an `ArrayList` of key-value pairs) — this is **separate chaining**. Average $O(1)$ holds as long as keys spread out roughly evenly across buckets, so chains stay short.

```
buckets (array, size = 16)

[0] -> (empty)
[1] -> (17,"x") -> (33,"y")    <- 17 % 16 == 1 and 33 % 16 == 1: collision, chained
[2] -> (2,"200")
[3] -> (empty)
...
[15] -> (31,"z")

put(17,"x"):  bucket = 17 % 16 = 1  -> append (17,"x") to bucket[1]
get(33):      bucket = 33 % 16 = 1  -> scan bucket[1] for key==33 -> "y"
remove(17):   bucket = 1           -> scan, drop the (17,"x") entry from that chain
```

Approach (step by step)

Key idea / invariant. Every key deterministically maps to exactly one bucket via `hash(key) = key % numBuckets` (using `Math.floorMod` so negative keys still land in a valid bucket). All keys that collide on the same bucket index live together in a small chain (a list of entries) *inside* that bucket. The invariant: at most one entry per bucket has a given key — `put` must check the chain before blindly appending, or duplicates would pile up.

1. **put(key, value)** : compute `idx = bucketIndex(key)`. Walk the chain at `buckets[idx]` looking for an existing entry with that key. If found, overwrite its value (update). If not found after scanning the whole chain, append a new `Entry(key, value)` to the chain.
2. **get(key)** : compute the same `idx`, walk the chain, return the value of the first entry whose key matches, or `-1` if the chain is exhausted without a match.

3. **remove(key)** : compute `idx` , then filter the entry with that key out of the chain (`removeIf`) — a no-op if it isn't there.
4. **Why $O(1)$ average:** the number of buckets (769, a prime, chosen to distribute hashes evenly and avoid patterns with power-of-two moduli) is picked large relative to the expected key count, so each chain is expected to hold only a handful of entries. The work per call is $O(1)$ for the bucket lookup plus $O(\text{chain length})$ for the scan — average $O(1)$ as long as keys are spread roughly uniformly and chains stay short. Worst case (all keys collide into one bucket) degrades to $O(n)$, which is exactly why real hash maps `resize/rehash` or upgrade a long chain to a balanced tree.

Worked trace (`numBuckets = 16` for a short example):

```
put(17,"x")    idx = 17 % 16 = 1    buckets[1] = [(17,"x")]
put(33,"y")    idx = 33 % 16 = 1    buckets[1] = [(17,"x"), (33,"y")]  (collision, chained
get(33)        idx = 1             scan chain: 17 != 33, 33 == 33 -> return "y"
remove(17)     idx = 1             scan chain, drop (17,"x") -> buckets[1] = [(33,"y")]
```

Implementation (array of buckets + chaining)

```
public class MyHashMap {
    private static class Entry {
        int key, value;
        Entry(int key, int value) { this.key = key; this.value = value; }
    }

    private final int numBuckets = 769;           // a prime number of buckets
    private final List<List<Entry>> buckets = new ArrayList<>();

    public MyHashMap() {
        for (int i = 0; i < numBuckets; i++) {
            buckets.add(new ArrayList<>());
        }
    }

    private int bucketIndex(int key) {
        return Math.floorMod(key, numBuckets);    // handles negative keys correctly
    }

    public void put(int key, int value) {
        List<Entry> chain = buckets.get(bucketIndex(key));
        for (Entry e : chain) {
            if (e.key == key) {
                e.value = value;                  // update existing
                return;
            }
        }
        chain.add(new Entry(key, value));         // insert new
    }

    public int get(int key) {
        List<Entry> chain = buckets.get(bucketIndex(key));
        for (Entry e : chain) {
            if (e.key == key) return e.value;
        }
        return -1;
    }

    public void remove(int key) {
        List<Entry> chain = buckets.get(bucketIndex(key));
        chain.removeIf(e -> e.key == key);
    }
}
```

Operation	Required	Achieved
put(key, value)	O(1) avg	O(1) avg, O(chain length) worst case
get(key)	O(1) avg	O(1) avg, O(chain length) worst case
remove(key)	O(1) avg	O(1) avg, O(chain length) worst case
Space	—	O(numBuckets + n)

If chains get long (many collisions), real implementations resize the bucket array and rehash, or switch a long chain to a balanced tree — that's exactly what Java's own `HashMap` does once a bucket's chain passes a threshold.

3.4 Time Based Key-Value Store

Problem. Implement a `TimeMap` class supporting: - `set(key, value, timestamp)` — store the (key, value) pair at the given timestamp. - `get(key, timestamp)` — return the value associated with key at the **largest recorded timestamp that is $\leq$ timestamp**; return `""` if no such timestamp exists for that key.

Constraints (typical LeetCode bounds): - $1 \leq \text{key.length}, \text{value.length} \leq 100$, lowercase English letters. - $1 \leq \text{timestamp} \leq 10^7$. - All calls to `set` for the same key use **strictly increasing** timestamps. - At most $2 * 10^5$ calls total to `set` and `get`.

Clarifying assumptions — confirm with the interviewer: - Because timestamps are strictly increasing per key, you never need to insert into the middle of that key's history or re-sort — appends stay in sorted order for free. - `get` on a key that was never `set`, or on a timestamp before the key's first `set`, both return `""`.

Why it's tricky: the naive instinct is to scan all recorded timestamps for a key to find the best one, which is $O(n)$ per `get`; the required $O(\log n)$ only falls out once you notice the per-key history is already sorted, turning this into "binary search for the rightmost value that is $\leq$ target" rather than a search for an exact match.

Example.

```
TimeMap map = new TimeMap();
map.set("foo", "bar", 1);
map.get("foo", 1);      -> "bar"
map.get("foo", 3);      -> "bar"    (no entry at 3, most recent <= 3 is at t=1)
map.set("foo", "bar2", 4);
map.get("foo", 4);      -> "bar2"
map.get("foo", 5);      -> "bar2"    (most recent <= 5 is at t=4)
map.get("foo", 0);      -> ""       (nothing recorded yet at t=0)
```

Intuition (diagram). Store, per key, a **list of (timestamp, value) pairs** — timestamps for a given key only increase, so the list is already sorted by construction; no need to sort later. Use a **HashMap** for $O(1)$ key lookup, then **binary search** the list for the rightmost timestamp that is $\leq$ target. This is the classic "binary search on the answer": instead of searching for an exact match, you're searching for the boundary between "too far in the future" and "valid," and taking the last valid one.

```
map["foo"] -> index:  0          1          2
                pairs: (t=1,"bar") (t=4,"bar2") (t=9,"bar3")
                      ^ timestamps strictly increasing left to right

get("foo", 5):  binary search for largest t <= 5
                lo=0 hi=2
                mid=1: t=4 <= 5 -> candidate=1, search right half (lo=2)
                mid=2: t=9 > 5 -> search left half (hi=1)
                lo > hi, stop -> best candidate index = 1 -> return "bar2"
```

Approach (step by step)

Key idea / invariant. Because a given key's timestamps are guaranteed strictly increasing, its list of `(timestamp, value)` pairs is *already sorted by timestamp* the moment you always append (never insert in the middle). That turns `get` into a classic **binary search for the rightmost element $\leq$ target** — not a search for an exact match, but a search for a boundary: everything to the left of the boundary is a valid candidate ("timestamp is in the past or present"), everything to the right is invalid ("timestamp is in the future").

1. `set(key, value, timestamp)` : look up (or create) the list for `key` via `computeIfAbsent` , and append `(timestamp, value)` . $O(1)$ — no sorting needed because the caller guarantees increasing timestamps.
2. `get(key, timestamp)` : fetch the key's list; if missing or empty, return `""` . Otherwise binary search it:
3. Maintain `lo` , `hi` bounding the search range, and `bestIdx` for the best boundary found so far (`-1` = none yet).
4. At each `mid` , if `pairs[mid].timestamp <= timestamp` , `mid` is a *valid* candidate — but a later index might be an even better (larger, still valid) one, so record `bestIdx = mid` and search the right half (`lo = mid + 1`).
5. Otherwise `mid` 's timestamp is too far in the future — search the left half (`hi = mid - 1`).
6. When `lo > hi` , `bestIdx` holds the rightmost valid index (or stays `-1` if nothing qualified), giving $O(\log n)$ instead of an $O(n)$ linear scan.
7. **Why $O(\log n)$ / $O(1)$:** `set` never scans or shifts existing elements (append only); `get` 's binary search halves the search range each iteration over that one key's list, so it costs $O(\log n)$ where n is the number of times `set` was called for that key — not the total number of keys or total `set` calls overall.

Worked trace — `get("foo", 5)` on `pairs = [(1,"bar"), (4,"bar2"), (9,"bar3")]` :

```
lo=0 hi=2 bestIdx=-1
mid=1 pairs[1].timestamp=4 <= 5 -> bestIdx=1, lo=2    (keep looking right for a tighter f
mid=2 pairs[2].timestamp=9 > 5  -> hi=1              (too far in the future, go left)
lo=2 > hi=1, stop -> return pairs[bestIdx=1].value = "bar2"
```

Implementation (HashMap<key, List<(timestamp,value)>> + binary search)

```
public class TimeMap {
    private static class Pair {
        int timestamp;
        String value;
        Pair(int timestamp, String value) { this.timestamp = timestamp; this.value = value; }
    }

    private final Map<String, List<Pair>> store = new HashMap<>();

    public void set(String key, String value, int timestamp) {
        // timestamps are strictly increasing per key, so this stays sorted for free
        store.computeIfAbsent(key, k -> new ArrayList<>()).add(new Pair(timestamp, value));
    }

    public String get(String key, int timestamp) {
        List<Pair> pairs = store.get(key);
        if (pairs == null || pairs.isEmpty()) return "";

        int lo = 0, hi = pairs.size() - 1, bestIdx = -1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (pairs.get(mid).timestamp <= timestamp) {
                bestIdx = mid; // valid candidate, but a later one might also be valid
                lo = mid + 1; // keep looking to the right for a bigger valid timestamp
            } else {
                hi = mid - 1; // mid is too far in the future, look left
            }
        }
        return bestIdx == -1 ? "" : pairs.get(bestIdx).value;
    }
}
```

Operation	Required	Achieved
set(key, value, timestamp)	O(1) amortized	O(1) — map lookup + list append
get(key, timestamp)	O(log n)	O(log n) — map lookup + binary search over n entries for that key
Space	—	O(total number of set calls)

Closing notes

- State each operation's required complexity *before* writing any code — it dictates which primitives are even viable.
- Pick primitives to hit those bounds: hash map → O(1) lookup by key; doubly linked list → O(1) removal/insertion at a known node while preserving order; array/ArrayList → O(1) random access by index.
- For Union-Find, remember **path compression** and **union by rank/size** — without both, operations degrade toward O(n) instead of amortized O(1).

- Dijkstra's algorithm requires **non-negative edge weights**; negative weights need Bellman-Ford instead.
 - A greedy approach isn't "correct because it feels right" — justify it with an **exchange argument** (show swapping any two choices never improves the result).
 - Always test the **eviction path** and the **empty structure** case: LRU eviction on a freshly-full cache, `getRandom()` / `get()` on an empty set or map, and `get()` for a timestamp before any `set()` has happened.
-